APEXFORMZ V2

Product Requirements Document

Modular Human-Motion Intelligence + Sport-Specific Expert Engines

Document Status: Master Product & Engineering PRD
Version: 2.0
Date: September 2026
Product: ApexFormz / SmartCoach
Primary Objective: Transform the existing multi-sport pose-analysis architecture into a scalable, modular sports-intelligence platform built around a shared Human Motion Engine and isolated Sport Expert Engines.

---

1. EXECUTIVE SUMMARY

ApexFormz currently operates as a real-time biomechanical coaching platform built around:

- browser-based MediaPipe pose extraction,
- a Next.js user interface,
- Node.js/Express orchestration,
- Socket.IO real-time transport,
- a FastAPI AI-analysis service,
- static pose analysis,
- dynamic phase-based movement analysis,
- nearest-neighbour expert-frame matching,
- DTW trajectory analysis,
- kinetic-chain validation,
- signal filtering,
- ST-GCN Action Quality Assessment,
- PostgreSQL persistence,
- gamification,

- and social training systems.

The current platform already separates static and dynamic movement analysis and maintains movement definitions through sport/movement configuration.
However, ApexFormz is now moving from a controlled prototype architecture toward a real-world multi-sport platform.

The next architectural challenge is **domain isolation**.

Sports may contain superficially similar body configurations while requiring very different biomechanical interpretation.

Examples:

- badminton overhead forehand vs tennis serve,
- badminton forehand vs tennis forehand,
- cricket bowling vs throwing,
- boxing rotation vs racquet-sport rotation,
- Taekwondo kicking vs football kicking.

A universal movement reference database can therefore create semantic ambiguity even when the pose geometry itself is correctly measured.

The V2 architecture solves this through hierarchical intelligence:

CAMERA / VIDEO

↓

PERCEPTION CORE

↓

UNIVERSAL HUMAN MOTION ENGINE

↓

SPORT CONTEXT / SPORT ROUTER

↓

SPORT EXPERT ENGINE

↓

MOVEMENT ROUTER

↓

MOVEMENT EXPERT ENGINE

↓

BIOMECHANICAL ANALYSIS

↓

COACHING INTELLIGENCE

↓

ATHLETE INTELLIGENCE

The Universal Human Motion Engine understands the human body.

The Sport Engine understands the sport.

The Movement Engine understands the specific movement.

The Coaching Engine explains what should change.

The Athlete Intelligence layer understands the athlete over time.

This modular architecture becomes the foundation for future ApexFormz sports.

---

2. PRODUCT VISION

ApexFormz should become:

A modular human-performance intelligence platform capable of understanding how an athlete moves, interpreting that movement within the correct sport, evaluating it against validated expert execution, and delivering real-time personalized coaching.

The platform should not merely answer:

“Where is the athlete's knee?”

It should eventually answer:

“During the acceleration phase of your badminton smash, your shoulder initiated before your hip rotation, reducing kinetic-chain efficiency. On your previous eight reps, this sequencing problem occurred six times.”

The long-term ApexFormz intelligence hierarchy is therefore:

Layer 1 — Perception

What does the camera observe?

Layer 2 — Human Motion

What is the body physically doing?

Layer 3 — Sport Understanding

What does this body movement mean within badminton, tennis, cricket, Taekwondo, etc.?

Layer 4 — Movement Understanding

Which exact movement and phase is being performed?

Layer 5 — Biomechanical Evaluation

How does this execution differ from accepted expert execution?

Layer 6 — Coaching

What correction should the athlete make?

Layer 7 — Athlete Intelligence

What recurring strengths, weaknesses, improvements and patterns exist for this athlete?

---

3. PROBLEM STATEMENT

3.1 Current scaling problem

The current dynamic architecture supports multiple sports and movements through movement configuration and routing. Dynamic sessions maintain their own state and the FrameMatcher uses expert reference frames for movement comparison.

This is appropriate for the current stage.

However, as the reference database grows, the platform needs stronger architectural guarantees that:

- badminton data cannot affect tennis scoring,
- tennis references cannot affect Taekwondo analysis,
- movement-specific rules cannot leak between movements,

- sport-specific biomechanics remain isolated,
- sport-specific models can be developed independently,
- scoring criteria can differ between sports,
- future ML models can use shared representations without sharing incorrect decision boundaries.

The problem is not simply database size.

The fundamental problem is **semantic contamination**.

A human configuration can be geometrically similar across sports while carrying different biomechanical meaning.

---

4. PRODUCT GOALS

4.1 Primary goals

ApexFormz V2 shall:

1. Introduce a universal Human Motion Engine that contains only sport-independent movement intelligence.
2. Introduce isolated Sport Expert Engines.
3. Introduce isolated Movement Expert Engines within each sport.
4. Prevent cross-sport reference matching by architecture.
5. Support different scoring systems for different sports and movements.
6. Allow sport engines to request specific motion features from the Core Engine.
7. Preserve the existing real-time latency architecture.
8. Reuse existing DynamicAnalyzer capabilities wherever appropriate.
9. Support sport-specific reference datasets.
10. Support sport-specific ML models in the future.
11. Support explicit sport selection immediately.
12. Support optional automatic sport/movement recognition later.

13. Make adding a new sport primarily a plugin/configuration/data task rather than requiring changes to the central analysis engine.
14. Build a foundation for longitudinal athlete intelligence.

---

5. NON-GOALS FOR INITIAL V2

The initial architecture migration will NOT require:

- fully autonomous sport recognition,
- fully autonomous movement recognition for every sport,
- replacing MediaPipe,
- replacing the Next.js frontend,
- replacing the Node backend,
- replacing FastAPI,
- rewriting gamification,
- training a universal deep-learning sports model,
- multi-person tracking,
- mobile-native applications,
- fully automated coach certification,
- real-time ball/shuttle/racket tracking for all sports.

These may be added after the modular engine foundation is stable.

---

6. CURRENT PLATFORM BASELINE

The existing platform uses three major application services:

apps/frontend

apps/backend

apps/ai-service

with shared TypeScript types and PostgreSQL persistence.

6.1 Existing frontend

The frontend currently:

- captures webcam video,
- initializes MediaPipe Pose Landmarker,
- extracts body landmarks,
- opens authenticated Socket.IO sessions,
- emits frame data,
- receives analysis results,
- displays scores,
- displays issues,
- displays phase,
- displays rep count,
- renders the body skeleton.

6.2 Existing backend

The Node backend currently:

- manages authentication,
- manages training sessions,
- determines static/dynamic routing,
- relays frames,
- uses keep-alive connections to the AI service,
- drops stale queued frames,
- persists results,
- manages XP,
- manages streaks,
- manages leaderboards,
- manages Circles.

The existing:

frameInFlight

pendingFrame

strategy must be retained.

Real-time movement coaching must prioritize the newest frame rather than processing every historical frame.

6.3 Existing AI capabilities

Existing AI components include:

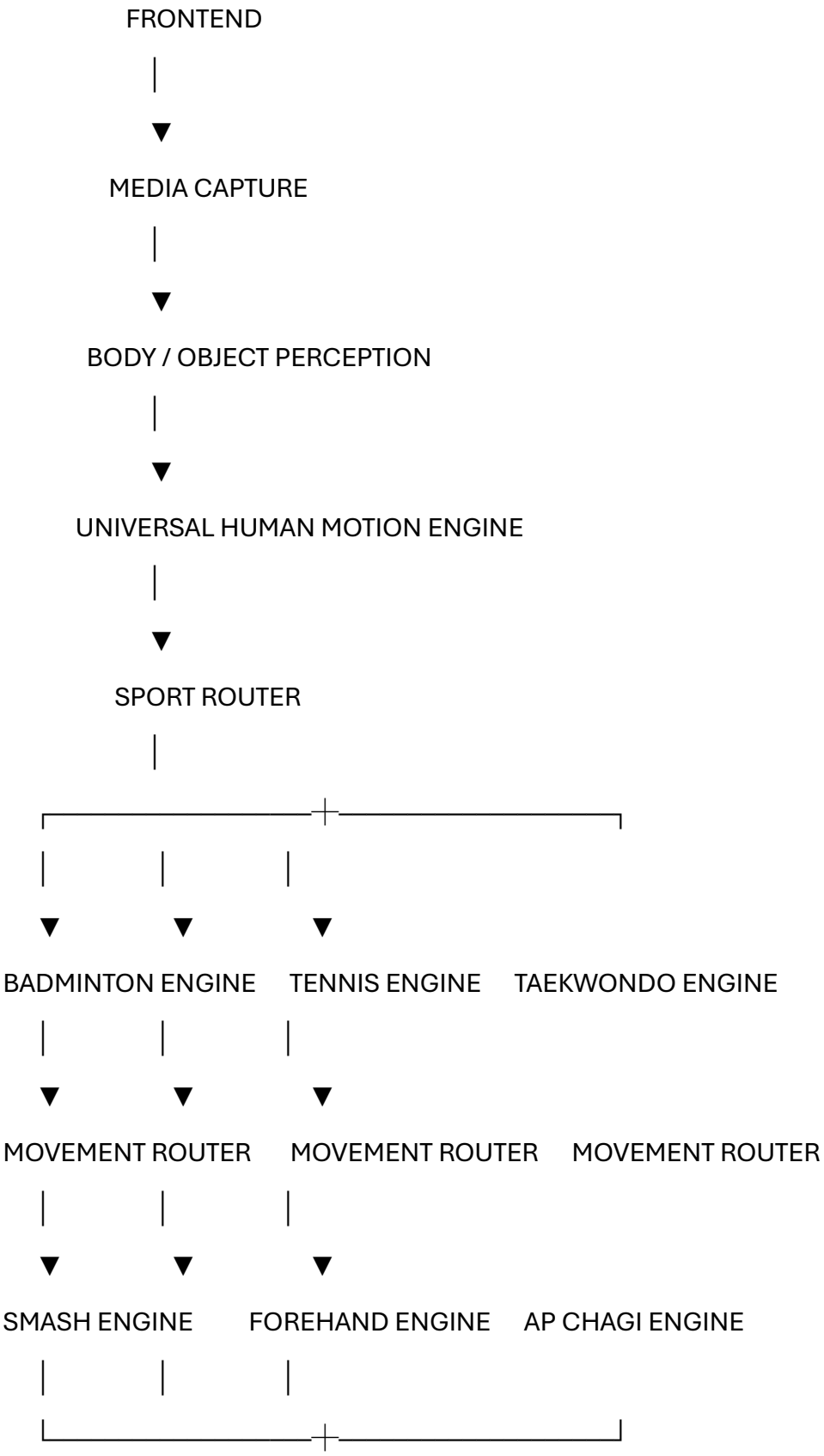
- static pose analyzer,
- BaseAnalyzer,
- AnalyzerRegistry,
- DynamicAnalyzer,
- PhaseDetector,
- PyomecaStreamAdapter,
- One Euro KeypointSmoother,
- FrameMatcher,
- KineticChainValidator,
- TrajectoryScorer,
- MovementRegistry,
- ST-GCN AQA,
- expert-frame data,
- movement blueprints.

These components should be reorganized—not discarded.

---

7. TARGET SYSTEM ARCHITECTURE

The target ApexFormz architecture shall be:


FRONTEND
MEDIA CAPTURE
BODY / OBJECT PERCEPTION
UNIVERSAL HUMAN MOTION ENGINE
SPORT ROUTER
BADMINTON ENGINE
TENNIS ENGINE
TAEKWONDO ENGINE
MOVEMENT ROUTER
MOVEMENT ROUTER
MOVEMENT ROUTER
SMASH ENGINE
FOREHAND ENGINE
AP CHAGI ENGINE

▼

BIOMECHANICS ENGINE

▼

COACHING ENGINE

▼

ATHLETE INTELLIGENCE

▼

BACKEND

▼

DATABASE

---

8. CORE DESIGN PRINCIPLE

The Core Engine must contain **measurements**.

The Sport Engine must contain **meaning**.

Example:

Core Engine output:

```
{
  "right_knee_angle": 104,
  "right_knee_angular_velocity": -187,
  "hip_rotation_velocity": 121,
  "shoulder_rotation_velocity": 203
}
```

The Core Engine must NOT conclude:

"Bad badminton smash"

The Badminton Smash Engine may interpret those measurements as:

shoulder rotation began before hip rotation

and the Coaching Engine may produce:

Initiate the stroke from your hips before accelerating your shoulder.

This separation is mandatory.

---

9. MODULE 01 — PERCEPTION CORE

9.1 Purpose

Convert visual input into structured observations.

9.2 V2 input sources

Initial:

- webcam,
- uploaded video.

Future:

- mobile camera,
- multi-camera,
- prerecorded coach footage,
- external camera devices.

9.3 Current implementation

MediaPipe Pose Landmarker currently extracts body landmarks in-browser.

This should remain the default V2 implementation.

9.4 Required output

The perception layer shall produce:

```
interface RawPoseObservation {
  timestampMs: number;
  frameId: string;

  landmarks: Landmark[];
```

```
  detectionConfidence: number;


  source: {
    width: number;
    height: number;
    fpsEstimate: number;
  };
}
```

Landmark must include:

```
interface Landmark {
  name: string;
  x: number;
  y: number;
  z?: number;
  visibility: number;
}
```

Critical requirement

The V2 data contract must explicitly preserve z whenever the underlying pose detector provides it.

The current frontend documentation demonstrates a type containing x/y without z while AI documentation describes optional 3D processing.
The V2 implementation must verify and standardize this.

---

10. MODULE 02 — UNIVERSAL HUMAN MOTION ENGINE

10.1 Purpose

Transform raw landmarks into a standardized, sport-neutral representation of human motion.

10.2 Responsibilities

The engine shall calculate, where technically possible:

Skeleton geometry

- joint positions,
- body centre,
- shoulder width,
- hip width,
- limb lengths,
- torso vector.

Joint geometry

- elbow angle,
- shoulder angle,
- hip angle,
- knee angle,
- ankle angle where supported,
- torso inclination,
- shoulder-line orientation,
- hip-line orientation.

Motion derivatives

- position velocity,
- joint angular velocity,
- acceleration,
- angular acceleration,
- jerk.

Temporal information

- movement active/inactive,
- temporal history,
- onset detection,
- movement termination,
- approximate repetition boundaries.

Body orientation

- left/right orientation,
- body-facing direction,
- vertical/horizontal orientation,
- stance direction.

Signal confidence

Every feature must contain a confidence value where appropriate.

---

11. UNIVERSAL MOTION REPRESENTATION

The Human Motion Engine shall return a MotionFrame.

Example:

```
interface MotionFrame {

  timestampMs: number;

  landmarks: NormalizedLandmarks;

  jointAngles: Record<string, number>;

  jointAngularVelocity: Record<string, number>;
```

```
jointAngularAcceleration: Record<string, number>;

positionVelocity: Record<string, Vector3>;

body: {
  torsoLean: number;
  shoulderRotation: number;
  hipRotation: number;
  shoulderHipSeparation: number;
  centerOfBody: Vector3;
};

orientation: {
  vertical: boolean;
  facingDirection?: string;
  dominantSideEstimate?: "left" | "right" | "unknown";
};

confidence: {
  tracking: number;
  visibility: number;
};

motionState: {
  active: boolean;
```

```
    onsetProbability?: number;
  };
}
```

This object becomes the primary input to all sport engines.

---

12. SIGNAL PROCESSING

The existing One Euro filtering and optional Pyomeca/Butterworth pipeline should become services available to the Human Motion Engine.

Proposed structure:

```
core/
  signal/
    one_euro.py
    butterworth.py
    derivative.py
    interpolation.py
```

Sport engines should not independently implement smoothing unless they have a documented special requirement.

---

13. FEATURE REGISTRY

ApexFormz should introduce a Feature Registry.

Example:

Feature Registry

joint_angle.right_knee

joint_velocity.right_knee

torso.rotation

hip.rotation

shoulder.rotation

center.vertical_velocity

body.orientation

Sport engines declare which features they require.

Example:

```
{
  "sport": "badminton",
  "required_features": [
    "shoulder.rotation",
    "hip.rotation",
    "joint_angle.right_elbow",
    "joint_velocity.right_elbow"
  ]
}
```

This creates a stable contract between universal motion processing and sport-specific intelligence.

---

14. MODULE 03 — SPORT ROUTER

14.1 Purpose

Determine which Sport Expert Engine receives the MotionFrame.

14.2 V2 Phase 1

Sport is explicitly selected by the user.

Example:

/train/badminton

creates:

{

```
  "sport": "badminton"
}
```

The backend/AI service then resolves:

```
SPORT_ENGINE_REGISTRY["badminton"]
```

No other sport engine may participate in scoring.

14.3 Future automatic recognition

Future implementation may return:

```
{
  "badminton": 0.91,
  "tennis": 0.05,
  "squash": 0.03,
  "unknown": 0.01
}
```

Automatic routing must use confidence thresholds.

If confidence is insufficient:

```
sport_status = "uncertain"
```

The system must not silently select a sport.

---

15. SPORT ENGINE REGISTRY

Introduce:

```
SPORT_ENGINE_REGISTRY = {
    "taekwondo": TaekwondoEngine,
    "badminton": BadmintonEngine,
    "tennis": TennisEngine,
    "cricket": CricketEngine,
}
```

Every registered engine must satisfy the same contract.

Example:

```
class SportEngine(ABC):


  @abstractmethod
  def supported_movements(self):
    pass


  @abstractmethod
  def create_session(self, config):
    pass


  @abstractmethod
  def classify_movement(self, motion_window):
    pass


  @abstractmethod
  def get_movement_engine(self, movement_name):
    pass


  @abstractmethod
  def required_features(self):
    pass
```

---

16. MODULE 04 — SPORT EXPERT ENGINE

Each sport engine owns:

- terminology,
- movement inventory,
- biomechanical rules,
- reference dataset,
- allowed phase models,
- movement classifier,
- scoring policies,
- sport-level coaching logic,
- sport-specific models,
- equipment requirements.

It must NOT own:

- generic landmark extraction,
- generic angle mathematics,
- generic signal filtering,
- authentication,
- session persistence.

---

17. PROPOSED DIRECTORY STRUCTURE

Recommended AI-service refactor:

apps/ai-service/app/

core/

├— perception/

├— motion/

│ ├— normalization.py

│ ├— angles.py

```
│   ├── velocity.py
│   ├── acceleration.py
│   ├── orientation.py
│   └── motion_frame.py
│
├── signal/
│   ├── one_euro.py
│   ├── butterworth.py
│   └── interpolation.py
│
├── features/
│   ├── registry.py
│   └── extractor.py
│
└── confidence/
    └── tracking_quality.py



sports/
├── base/
│   ├── sport_engine.py
│   ├── movement_engine.py
│   └── contracts.py
│
├── taekwondo/
```

```
│   ├── engine.py
│   ├── movements/
│   │   ├── ap_chagi.py
│   │   ├── dollyo_chagi.py
│   │   └── ...
│   ├── data/
│   ├── models/
│   └── rules/
│
├── badminton/
│   ├── engine.py
│   ├── movements/
│   ├── data/
│   ├── models/
│   └── rules/
│
├── tennis/
│   └── ...
│
└── cricket/
    └── ...


analysis/
├── phase/
```

├── matching/

├── trajectory/

├── kinetic_chain/

├── scoring/

└── coaching/

athlete/

├── profiling/

├── trends/

├── weakness_detection/

└── personalization/

---

18. MODULE 05 — MOVEMENT ROUTER

Within each sport, the Sport Engine selects the specific movement.

Example:

Badminton Engine

    │

    ├── smash

    ├── clear

    ├── drop

    ├── drive

    ├── serve

    └── net shot

There are two operating modes.

Explicit movement

User selects:

Badminton → Smash

This should remain the default early architecture.

Automatic movement recognition

Later:

```
{
  "smash": 0.87,
  "clear": 0.08,
  "drop": 0.03,
  "unknown": 0.02
}
```

Confidence gates are mandatory.

---

19. MOVEMENT ENGINE CONTRACT

Each movement engine should implement something similar to:

```
class MovementEngine(ABC):

    movement_id: str

    @abstractmethod
    def required_features(self):
        pass

    @abstractmethod
    def process_frame(self, motion_frame):
```

```
    pass

  @abstractmethod
  def detect_phase(self, motion_frame):
    pass

  @abstractmethod
  def score_frame(self, motion_frame):
    pass

  @abstractmethod
  def complete_rep(self):
    pass

  @abstractmethod
  def generate_analysis(self):
    pass
```

Movement engines may internally use shared ApexFormz analysis modules.

---

20. EXISTING DYNAMICANALYZER MIGRATION

The existing DynamicAnalyzer currently coordinates approximately ten analysis stages, including readiness, normalization, filtering, phase detection, scoring, trajectory recording, fluidity, confidence and overall scoring.

V2 should decompose this responsibility.

Current:

DynamicAnalyzer

  owns almost everything

V2:

MovementEngine

```
    │
    ├── PhaseDetector
    ├── ReferenceMatcher
    ├── TrajectoryScorer
    ├── KineticChainValidator
    ├── FluidityScorer
    └── MovementScorePolicy
```

DynamicAnalyzer can temporarily remain as an adapter during migration.

---

21. MODULE 06 — PHASE ENGINE

The current FSM-based PhaseDetector should become a reusable framework rather than being conceptually tied to one global dynamic engine.

Each movement owns its phase definition.

Example:

Badminton Smash

0 Ready

1 Preparation

2 Loading

3 Acceleration

4 Contact

5 Follow-through

6 Recovery

Taekwondo may define:

0 Stance

1 Chamber

2 Extension

3 Rechamber

4 Return

Phase configuration must remain movement-specific.

---

22. PHASE-AWARE REFERENCE MATCHING

This is a major V2 requirement.

Current expert frame matching uses expert movement frames and maintains a separate stance pool.

V2 must support:

movement

├— phase_0 reference frames

├— phase_1 reference frames

├— phase_2 reference frames

├— phase_3 reference frames

When athlete phase = acceleration:

search only acceleration reference frames

not:

search every frame in the entire movement

This reduces semantic ambiguity even inside the same movement.

---

23. REFERENCE DATA ARCHITECTURE

Recommended hierarchy:

reference_data/

badminton/

smash/

manifest.json

front/

side/

rear/

phase_index.json

trajectories.json

phase_centroids.json

clear/

drop/

tennis/

forehand/

backhand/

serve/

taekwondo/

ap_chagi/

dollyo_chagi/

---

24. REFERENCE MANIFEST

Every movement dataset must contain metadata.

Example:


```
{
 "sport": "badminton",
 "movement": "smash",

 "version": "1.0",

 "athletes": 18,

 "recordings": 91,

 "camera_views": [
  "front",
  "side",
  "rear"
 ],

 "dominant_hand": [
  "right",
  "left"
 ],

 "phases": [
  "ready",
  "preparation",
```

```
  "loading",
  "acceleration",
  "contact",
  "follow_through",
  "recovery"
 ]
}
```

---

25. REFERENCE DATA VERSIONING

Reference datasets must be immutable once released.

Example:

badminton-smash:v1.0

badminton-smash:v1.1

badminton-smash:v2.0

Training sessions must record which dataset version produced the score.

This is essential for reproducibility.

---

26. REFERENCE QUALITY CONTROL

Expert data should eventually contain:

- athlete identity anonymized ID,
- expertise classification,
- recording camera,
- dominant side,
- movement type,
- valid/invalid sample label,
- recording quality,

- phase annotations,
- coach approval,
- reference confidence.

Not every "expert" video should automatically become a canonical reference.

---

27. MODULE 07 — REFERENCE MATCHER

The current FrameMatcher uses weighted nearest-neighbour comparison against expert joint-angle frames.

V2 should generalize it into:

ReferenceMatcher

with retrieval constraints:

SPORT

AND MOVEMENT

AND PHASE

AND optionally:

VIEW

DOMINANT SIDE

ATHLETE PROFILE RANGE

Example query:

sport = badminton

movement = smash

phase = acceleration

view = side

dominant_hand = right

Then nearest-neighbour matching happens only within this filtered domain.

---

28. FUTURE REFERENCE INDEXING

The existing matcher performs brute-force O(N) search.

As data grows, implement an abstraction supporting:

Initial:

BruteForceIndex

Later:

KDTreeIndex

HNSWIndex

FAISSIndex

The movement engine should not care which search implementation is used.

---

29. FEATURE VECTOR

The reference vector should no longer be limited conceptually to eight angles.

Example:

angles

+

angular velocity

+

body orientation

+

hip rotation

+

shoulder rotation

+

torso lean

+

relative landmark geometry

+

centre displacement

Different movement engines specify feature weights.

---

30. MOVEMENT-SPECIFIC FEATURE WEIGHTS

Example Taekwondo:

hip rotation 1.8

knee extension 2.0

support knee 1.2

shoulder position 0.5

Example badminton smash:

shoulder rotation 1.7

elbow extension 1.5

hip rotation 1.4

torso rotation 1.4

wrist/forearm signal future

No universal joint-weight table should be assumed across every sport.

---

31. MODULE 08 — TRAJECTORY ENGINE

The existing DTW trajectory scorer should become a reusable movement-analysis service.

Input:

athlete movement trajectory

reference trajectory

feature selection

Output:

{
 "similarity": 84.1,
 "features_compared": [
  "right_knee",
  "right_hip"
 ],
 "confidence": 0.91
}

Movement engines determine:

- which joints/features matter,
- how trajectory contributes to total score.

---

32. MODULE 09 — KINETIC CHAIN ENGINE

The existing KineticChainValidator should be generalized.

Instead of hard-coded segment vocabulary, define a registry.

Example:

hips

torso

shoulders

elbows

wrists

knees

ankles

Movement config:

{
 "expected_chain": [

"hips",

"torso",

"shoulder",

"elbow",

"wrist"

]

}

The system calculates peak activation timing and evaluates ordering.

---

33. MODULE 10 — FLUIDITY ENGINE

The existing jerk-based fluidity logic may remain available as one metric.

However, fluidity must not automatically contribute the same percentage for every sport.

Example:

Yoga:

fluidity may be irrelevant

Badminton:

fluidity moderately relevant

Gymnastics:

fluidity highly relevant

Movement scoring policy determines weight.

---

34. MODULE 11 — AQA MODEL LAYER

The current ST-GCN runs asynchronously after rep completion.

V2 should convert this into a generic:

ActionQualityModel

interface.

Example:

```
class ActionQualityModel:


  def infer(
    self,
    trajectory,
    sport,
    movement
  ) -> AQAResult:
    ...
```

Each sport may eventually have:

taekwondo_aqa_v2

badminton_aqa_v1

tennis_aqa_v1

A universal AQA model should not be assumed.

---

35. NO RANDOM PRODUCTION SCORES

The current technical documentation notes that missing ST-GCN weights may produce randomized testing scores.

V2 production requirement:

If model unavailable:

```
{
 "aqa_status": "unavailable",
 "aqa_score": null
}
```

Never fabricate a score.

---

36. MODULE 12 — SCORING ENGINE

The scoring system must become configuration-driven.

Example schema:

```
{
  "movement": "ap_chagi",

  "components": {
    "phase_quality": 0.30,
    "trajectory": 0.25,
    "kinetic_chain": 0.20,
    "extension": 0.15,
    "balance": 0.10
  }
}
```

Another movement can use an entirely different formula.

---

37. SCORE OBJECT

Return more than a single overall score.

Example:

```
{
  "overall": 82,

  "components": {
    "phase_accuracy": 87,
    "trajectory": 79,
```

```
  "kinetic_chain": 66,
  "fluidity": 91,
  "balance": 84
 },

 "confidence": 0.92
}
```


The UI can still display one main number.

But ApexFormz must preserve the richer scoring information.

---

38. SCORE CONFIDENCE

A score without confidence can be misleading.

Return:


```
{
 "score": 84,
 "confidence": 0.94
}
```


Low confidence may result from:

- occlusion,
- poor lighting,
- incomplete body visibility,
- uncertain phase,
- ambiguous movement,
- unsupported camera angle.

When confidence falls below threshold, coaching should degrade safely.

---

39. MODULE 13 — COACHING ENGINE

The Coaching Engine transforms analysis into athlete-facing feedback.

Input:

movement analysis

+

phase

+

detected errors

+

error severity

+

athlete history

Output:

primary correction

secondary correction

positive observation

confidence

---

40. COACHING PRIORITY

Do not show ten simultaneous corrections.

Priority model:

Safety issue

↓

Major mechanical issue

↓

Major performance issue

↓

Persistent recurring issue

↓

Minor optimization

Recommended live UI:

1 primary correction

+

maximum 1 secondary correction

Detailed analysis may appear after the rep/session.

---

41. COACHING RULE FORMAT

Example:

```
{
  "rule_id": "badminton.smash.hip_lead",

  "condition": {
    "shoulder_peak_before_hip_peak_ms": "> 80"
  },

  "severity": "medium",

  "message": "Your shoulder is initiating before your hips.",

  "correction": "Start the rotation from the hips, then accelerate the shoulder."
}
```

Rules should remain sport/movement-specific.

---

42. MODULE 14 — ATHLETE INTELLIGENCE

This is the long-term personalization layer.

Current ApexFormz already stores sessions, frames, scores, XP and user statistics.

V2 should begin storing structured performance metrics.

Example:

Athlete

└── Badminton

    └── Smash

        ├── phase scores

        ├── kinetic chain

        ├── recurring issues

        ├── average score

        ├── best score

        ├── consistency

        └── trend

---

43. ATHLETE WEAKNESS MODEL

Example:

Over 10 smash reps:

hip initiation late

8/10 reps

elbow low

3/10 reps

recovery slow

5/10 reps

The Athlete Intelligence layer may conclude:

PRIMARY RECURRING ISSUE:

late hip initiation

The next session can focus coaching accordingly.

---

44. DATABASE CHANGES

Existing models should remain, including:

- User,
- TrainingSession,
- PoseFrame,
- UserStats,
- EnhancedStreak,
- Achievement,
- Leaderboard,
- Circle.

Add or extend models conceptually with:

Sport

Movement

MovementVersion

ReferenceDatasetVersion

RepAnalysis

MetricResult

DetectedIssue

AthleteMovementProfile

MovementProgress

---

45. REP ANALYSIS MODEL

Recommended model:

RepAnalysis

id

session_id

rep_number

sport

movement

movement_version

reference_dataset_version

start_timestamp

end_timestamp

overall_score

score_confidence

phase_scores JSON

component_scores JSON

trajectory_score

kinetic_chain_score

fluidity_score

aqa_score

issues JSON

This will be more useful than reconstructing everything from raw frames.

---

46. RAW FRAME STORAGE STRATEGY

Current architecture persists analyzed PoseFrame records.

At scale, full per-frame relational storage can become expensive.

V2 should introduce configurable persistence:

Mode A

Debug mode:

store every frame

Mode B

Production normal:

store sampled frames

+

phase transition frames

+

issue frames

+

rep trajectories

Mode C

Research:

store complete trajectory

This must be configurable per environment/session.

---

47. BACKEND SESSION CONTRACT

Existing:

session:start

frame:submit

feedback:result

session:stop

should remain for compatibility.

Extend session:start:

```
{
  "sessionId": "...",
  "sport": "badminton",
  "movement": "smash",

  "analysisMode": "dynamic",

  "dominantSide": "right",

  "cameraView": "front"
}
```

---

48. NEW FRAME RESULT CONTRACT

Recommended:

```
interface ApexAnalysisResult {

   sessionId: string;
```

```
sport: string;
movement: string;

phase: {
  name: string;
  index: number;
  confidence: number;
};

rep: {
  count: number;
  completed: boolean;
};

scoring: {
  overall: number;
  confidence: number;
  components: Record<string, number>;
};

issues: CoachingIssue[];

metrics: Record<string, number>;

tracking: {
  confidence: number;
```

```
    bodyVisible: boolean;
  };

  diagnostics?: {
    engineVersion: string;
    sportEngineVersion: string;
    movementEngineVersion: string;
    referenceDatasetVersion: string;
  };
}
```

---

49. AI SERVICE API

Recommended V2 endpoints:

POST /v2/analyze/frame

POST /v2/analyze/rep

POST /v2/session/start

POST /v2/session/end

GET /v2/sports

GET /v2/sports/{sport}/movements

GET /v2/engines/status

GET /v2/health

During migration, existing endpoints remain available.

---

50. SESSION STATE

The current DynamicAnalyzer session registry exists in process memory and therefore limits horizontal scaling.

V2 abstraction:

SessionStateStore

Development implementation:

InMemorySessionStateStore

Production target:

RedisSessionStateStore

The analysis engine must not depend directly on Python process memory.

---

51. SPORT ENGINE SESSION STATE

Each session should hold:

sport engine

movement engine

phase state

signal filters

trajectory

kinetic state

rep counter

issue history

score history

Example:

```
AnalysisSession {
  core_state,
  sport_session,
  movement_session
```

}

---

52. FRONTEND REQUIREMENTS

The frontend should preserve:

- webcam panel,
- body skeleton,
- live score,
- issue display,
- phase,
- rep count.

Add:

Sport indicator

BADMINTON

Movement indicator

SMASH

Engine state

Ready

Tracking

Movement detected

Analyzing

Confidence warning

Move slightly farther from the camera.

Rep breakdown

Smash #4

Overall 83

Timing 78

Kinetic 71

Trajectory 89

Recovery 86

---

53. FEEDBACK STABILIZATION

Existing dynamic analysis stabilizes issues across multiple frames.

Retain and extend.

Possible state:

candidate issue

confirmed issue

resolved issue

persistent issue

Do not flash feedback based on one noisy frame.

---

54. CAMERA ORIENTATION HANDLING

Movement references can be camera-sensitive.

Each session should have:

camera_view =

front

front-left

left

rear-left

rear

rear-right

right

front-right

unknown

Initially user may select view manually.

Later camera orientation may be estimated.

ReferenceMatcher should prefer compatible views.

---

55. HANDEDNESS / DOMINANT SIDE

Racquet and combat sports often differ by side.

Sport engine session should know:

dominant_side

Options:

left

right

unknown

Mirroring should remain supported where valid, but mirroring must be movement-aware.

A left-handed tennis forehand is not simply blindly interchangeable with all right-handed reference conditions.

---

56. EQUIPMENT INTELLIGENCE — FUTURE FOUNDATION

The architecture should support future equipment perception even if V2 does not implement it fully.

Generic object contract:

```
interface TrackedSportObject {
  type: string;
  position: Vector3;
  velocity?: Vector3;
  confidence: number;
```

}

Examples:

Badminton:

racket

shuttle

Tennis:

racket

ball

Cricket:

bat

ball

Football:

ball

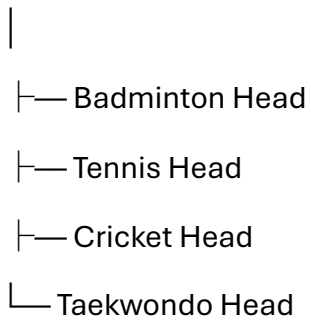
Sport engines may declare optional/required objects.

---

57. MODEL ARCHITECTURE — FUTURE

If learned representations are introduced, the preferred architecture is:

Shared Human Motion Encoder



This permits shared understanding of human motion while maintaining sport-specific decision boundaries.

Movement-specific heads may exist below sport heads.

---

58. VERSIONING

Every analysis result should eventually identify:

core_engine_version

sport_engine_version

movement_engine_version

reference_dataset_version

model_version

scoring_policy_version

Example:

```
{
  "core": "2.1.0",
  "sport_engine": "badminton-1.3.0",
  "movement_engine": "smash-2.0.1",
  "dataset": "badminton-smash-ref-1.4",
  "scoring": "smash-score-1.2"
}
```

This is essential for debugging and scientific evaluation.

---

59. OBSERVABILITY

Log per analysis session:

latency

FPS

tracking confidence

sport engine

movement engine

phase transitions

reference search latency

scoring latency

errors

dropped frames

Metrics:

P50 latency

P95 latency

P99 latency

frames/sec

AI service failures

phase confidence

movement confidence

---

60. PERFORMANCE REQUIREMENTS

The current target described by the project is approximately under 150 ms from frame capture to displayed feedback.

V2 should preserve this class of responsiveness.

Targets:

P50 AI analysis: < 60 ms

P95 AI analysis: < 120 ms

End-to-end feedback target: < 150 ms where environment permits

AQA and other expensive post-rep models must remain outside the live frame hot path.

---

61. FRAME BACKPRESSURE

Retain:

frameInFlight

+

latest pending frame

Never create an unlimited analysis queue.

The newest athlete state is more valuable than perfectly processing every camera frame.

---

62. CONFIDENCE GATES

Every important inference should support uncertainty.

Examples:

Tracking confidence

Low:

Do not provide precise biomechanical correction.

Movement confidence

Low:

Movement unclear. Perform again.

Phase confidence

Low:

Avoid phase-specific correction.

Reference confidence

Low:

General correction only.

The product must be designed to abstain when evidence is insufficient.

---

63. SAFETY BEHAVIOUR

ApexFormz should not present uncertain analysis as certainty.

Issue object:

{
"type": "hip_rotation",
"severity": "medium",
"confidence": 0.91,
"message": "Your hip rotation appears late."
}

Language can reflect uncertainty when necessary:

Your hip rotation appears to be starting late.

rather than falsely asserting exact biomechanics under poor visibility.

---

64. INITIAL SPORT MIGRATION

The first sport migrated into the modular architecture should be:

Taekwondo

because the existing project already has:

- ten dynamic movements,
- phase definitions,
- expert-video frame infrastructure,
- FrameMatcher support,
- trajectory scoring,
- AQA model,
- current validation work.

This makes Taekwondo the best reference implementation of a Sport Engine.

---

65. TAEKWONDO V2 STRUCTURE

Example:

TaekwondoEngine

├── ApChagiEngine

├── DollyoChagiEngine

├── YeopChagiEngine

├── DwiChagiEngine

└── remaining movements

Each movement receives:

phase definition

reference dataset

joint weights

kinetic chain

trajectory configuration

scoring configuration

coaching rules

---

66. SECOND SPORT

After the Taekwondo architecture is stable, implement either badminton or another chosen sport as the first **new native V2 sport**.

The goal is not only to support that sport.

The purpose is to prove that:

adding a new sport does not require rewriting ApexFormz Core.

---

67. SPORT PACKAGE DEFINITION

A sport package should ideally contain:

sport.yaml

movements/

references/

rules/

models/

tests/

Example:

sport: badminton

version: 1.0

supported_movements:

- smash
- clear
- drop

required_core_features:

- shoulder_rotation
- hip_rotation
- elbow_angle

---

68. MOVEMENT PACKAGE

Example:

sports/badminton/movements/smash/

definition.yaml

phases.yaml

scoring.yaml

kinetic_chain.yaml

coaching_rules.yaml

references/

tests/

This allows sports experts and engineers to work with explicit movement definitions.

---

69. TESTING STRATEGY

Each movement engine needs:

Unit tests

- angle processing,
- trigger logic,
- score calculation,
- reference filtering.

Recorded trajectory tests

Known trajectory:

expected phases

expected rep count

expected major issues

Regression tests

Reference engine version update must not unexpectedly alter baseline test scores.

Real athlete tests

Athlete performs:

correct movement

known incorrect movement A

known incorrect movement B

Validate output with a qualified coach.

---

70. GOLDEN MOVEMENT TEST SET

Create a permanent validation dataset.

Per movement:

20 correct examples

20 moderate examples

20 intentionally incorrect examples

Eventually increase substantially.

Each sample receives human expert annotations.

Automated evaluation then checks:

phase accuracy

rep detection

issue detection

score correlation

false positive rate

false negative rate

---

71. REQUIRED VALIDATION METRICS

Do not rely only on “the output looks good.”

Measure:

Movement recognition

precision

recall

F1

Phase detection

phase accuracy

transition timing error

Issue detection

issue precision

issue recall

Rep detection

rep-count accuracy

Scoring

Compare Apex score with expert coach scoring.

Metrics:

Pearson correlation

Spearman correlation

MAE

Reliability

Repeated similar movement should produce similar scores.

---

72. LATENCY TESTING

Measure separately:

MediaPipe latency

network latency

backend relay

core motion processing

sport-engine processing

reference lookup

response serialization

frontend render

This prevents unexplained end-to-end delays.

---

73. MIGRATION PLAN

Phase 1 — Core extraction

Move generic logic out of DynamicAnalyzer:

normalization

angles

filtering

orientation

velocity

confidence

Create Human Motion Engine.

No behavior change.

Phase 2 — Sport Engine interface

Create:

BaseSportEngine

SportEngineRegistry

MovementEngine

Wrap existing Taekwondo dynamic pipeline.

No major scoring changes.

Phase 3 — Taekwondo migration

Move:

Taekwondo movements

Taekwondo references

Taekwondo weights

Taekwondo phase definitions

Taekwondo AQA

inside:

sports/taekwondo/

Phase 4 — Movement-level isolation

Create individual Taekwondo Movement Engines.

Example:

ApChagiEngine

DollyoChagiEngine

Phase 5 — Phase-aware references

Update extraction pipeline to label expert frames by phase.

Matcher query becomes:

sport + movement + phase

Phase 6 — Score-policy architecture

Remove universal rep weighting assumptions.

Create movement-specific score configuration.

Phase 7 — Coaching Engine

Separate raw issue detection from athlete-facing language.

Phase 8 — Athlete Intelligence

Persist structured rep metrics.

Implement recurring weakness detection.

Phase 9 — Second sport

Implement BadmintonEngine or chosen sport entirely through the new plugin interface.

Success criterion:

No modification to Human Motion Engine should be necessary except adding genuinely universal features.

---

74. BACKWARD COMPATIBILITY

During migration:

Current endpoints:

/analyze

/analyze/dynamic

should remain operational.

Internally:

/analyze/dynamic

↓

LegacyDynamicAdapter

↓

SportEngineRegistry

Once migration is complete, frontend can move to /v2.

---

75. DEFINITION OF DONE — CORE ENGINE

The Core Engine is complete when:

- MediaPipe landmarks enter one common pipeline.
- 2D/3D representation is standardized.
- z values are preserved where available.
- generic angles work independently of sport.

- generic velocities work independently of sport.
- generic smoothing is centralized.
- generic confidence is centralized.
- no sport-specific terms exist inside the core package.
- automated tests pass.

---

76. DEFINITION OF DONE — SPORT ENGINE

A Sport Engine is complete when:

- registered through SportEngineRegistry,
- declares supported movements,
- loads only its own references,
- contains its own biomechanical definitions,
- contains its own movement list,
- supports independent versioning,
- can be disabled without affecting other sports,
- passes isolation tests.

---

77. ISOLATION TEST

Mandatory test:

Session sport = badminton

Then verify:

tennis reference repository access count = 0

taekwondo reference repository access count = 0

cricket reference repository access count = 0

This should be testable programmatically.

---

78. DEFINITION OF DONE — MOVEMENT ENGINE

Movement Engine complete when:

- phase model implemented,
- rep detection implemented,
- reference dataset connected,
- scoring policy configured,
- coaching rules configured,
- unit tests pass,
- golden dataset validation passes,
- latency within target.

---

79. ACCEPTANCE CRITERIA FOR V2 FOUNDATION

The architecture release is accepted when all of the following are true:

1. Existing Taekwondo analysis operates through TaekwondoEngine.
2. Human Motion Engine contains no Taekwondo-specific logic.
3. Movement references are sport-isolated.
4. Movement references are movement-isolated.
5. Phase-specific matching is supported.
6. Scoring policies are configurable by movement.
7. Coaching rules are separate from raw scoring.
8. Core and sport-engine versions appear in diagnostics.
9. Existing frontend continues to receive real-time feedback.
10. Existing authentication/gamification functionality continues working.
11. AI hot-path latency has not materially regressed.
12. No production system returns randomized AQA scores.
13. Automated tests verify sport isolation.

---

80. DEVELOPMENT PRIORITY

Recommended engineering order:

P0 — Architecture

- Human Motion Engine
- Sport Engine interface
- Movement Engine interface
- registry
- standardized frame contracts.

P0 — Reliability

- proper z-coordinate contract,
- confidence handling,
- production AQA fallback,
- session lifecycle,
- engine versioning.

P1 — Taekwondo migration

- move existing dynamic system into Sport Engine architecture,
- phase-specific reference organization,
- per-movement scoring.

P1 — Dataset architecture

- manifests,
- versioning,
- phase labels,
- expert metadata.

P1 — Validation

- golden datasets,

- coach annotations,
- automated scoring comparison.

P2 — Athlete intelligence

- recurring issue tracking,
- progression metrics,
- movement profiles.

P2 — New sport

- Badminton Engine.

P3 — Equipment perception

- racket,
- shuttle,
- ball,
- bat.

P3 — automatic recognition

- sport classifier,
- movement classifier.

---

81. MAJOR ENGINEERING RISKS

Risk 1 — False precision

Computer vision may generate precise numbers that are not biomechanically reliable.

Mitigation:

confidence

view awareness

visibility gates

uncertainty-aware coaching

Risk 2 — Dataset quality

A poor expert dataset makes sophisticated algorithms unreliable.

Mitigation:

expert validation

dataset versioning

multiple athletes

multiple camera views

controlled recordings

Risk 3 — Over-generalization

Trying to make one algorithm fit every sport can destroy sport-specific accuracy.

Mitigation:

Sport Engine architecture

Movement Engine architecture

Risk 4 — excessive specialization

Duplicating generic logic across sport engines creates maintenance problems.

Mitigation:

Universal Human Motion Engine

shared analysis primitives

clear contracts

Risk 5 — reference dataset growth

Brute-force nearest-neighbour matching will eventually become expensive.

Mitigation:

hierarchical filtering

phase filtering

ANN indexes

---

82. CORE PRODUCT PRINCIPLE

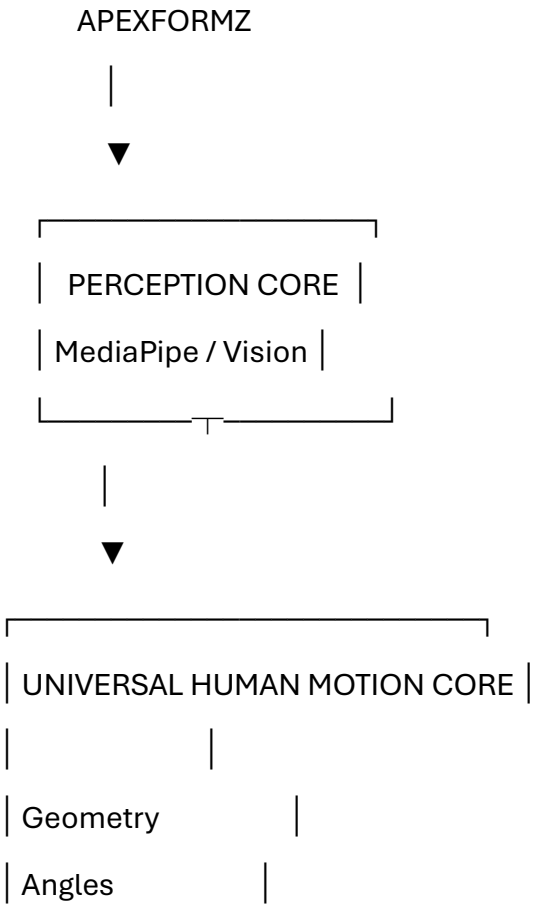
ApexFormz should follow:

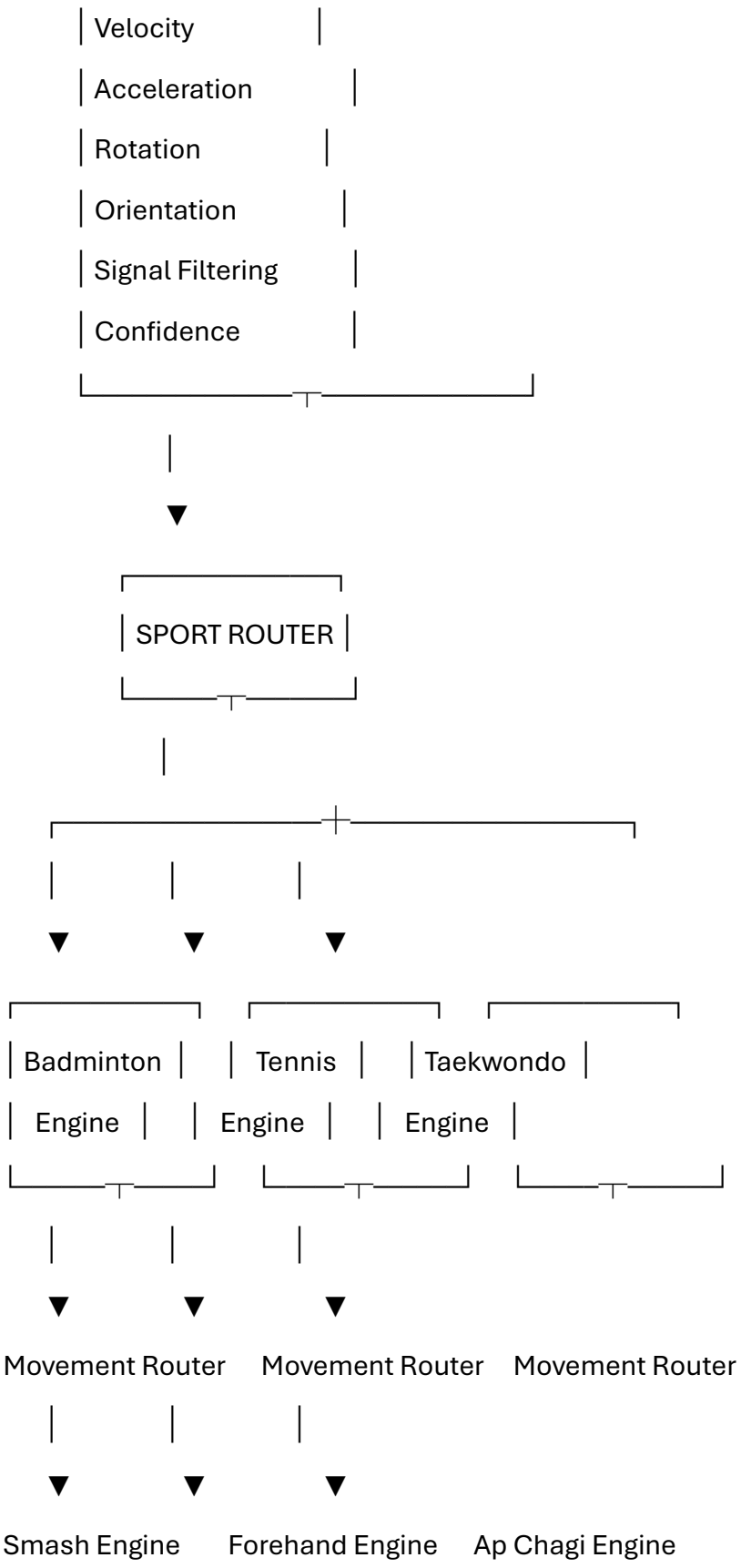
Share perception.
Share human-motion mathematics.
Isolate sport semantics.
Isolate movement knowledge.
Personalize coaching.

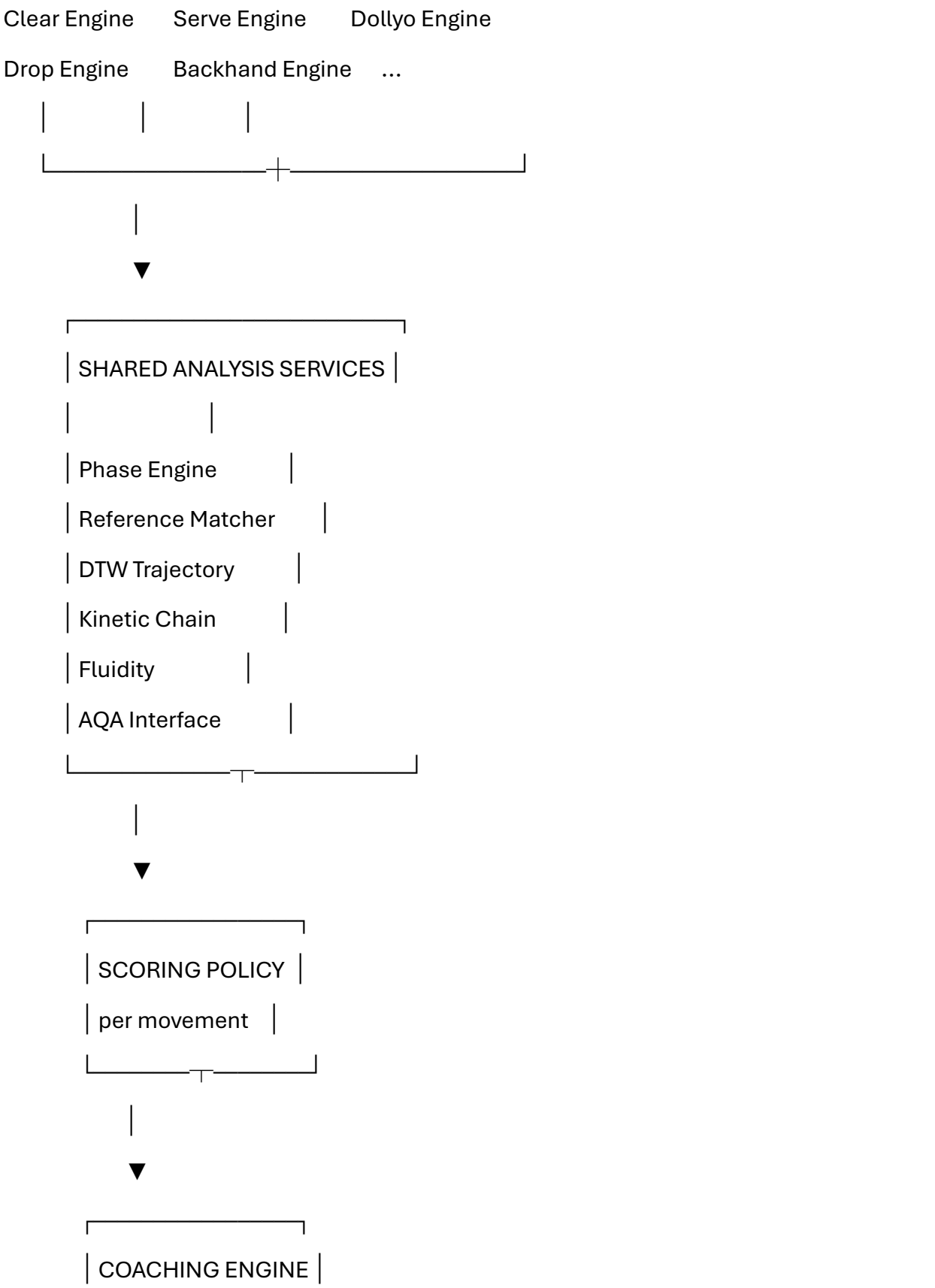
That principle should guide future architectural decisions.

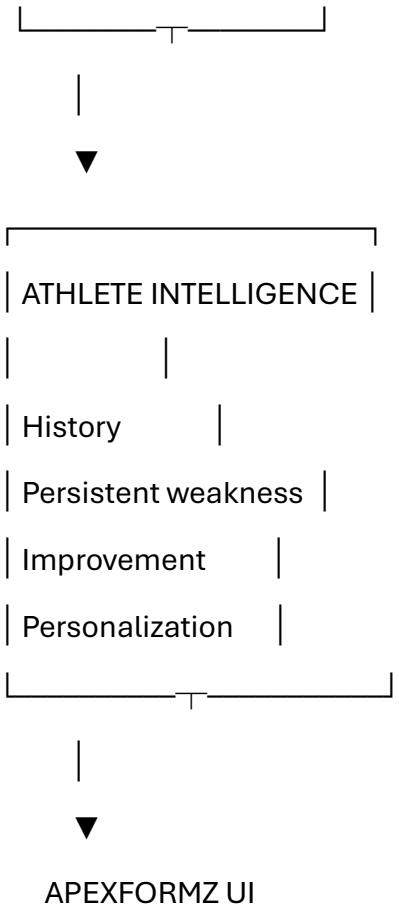
---

83. FINAL TARGET ARCHITECTURE




Velocity
Acceleration
Rotation
Orientation
Signal Filtering
Confidence
SPORT ROUTER
Badminton
Engine
Tennis
Engine
Taekwondo
Engine
Movement Router
Movement Router
Movement Router
Smash Engine
Forehand Engine
Ap Chagi Engine


Clear Engine
Serve Engine
Dollyo Engine
Drop Engine
Backhand Engine
...
SHARED ANALYSIS SERVICES
Phase Engine
Reference Matcher
DTW Trajectory
Kinetic Chain
Fluidity
AQA Interface
SCORING POLICY
per movement
COACHING ENGINE



---

84. PRODUCT OUTCOME

After this transition, ApexFormz will no longer be structured conceptually as:

one pose-analysis engine containing many sports.

It becomes:

a universal human-motion intelligence platform hosting independently evolving sport-specific coaching engines.

This provides:

- stronger domain accuracy,
- reduced cross-sport confusion,
- better dataset organization,
- independent sport development,

- sport-specific biomechanics,
- sport-specific scoring,
- sport-specific ML models,
- easier testing,
- easier scaling,
- easier scientific validation,
- clearer product extensibility,
- and a stronger long-term technical moat.

The architecture can then grow from:

Taekwondo

to:

Taekwondo

Badminton

Tennis

Cricket

Boxing

Running

Football

Yoga

Strength Training

without turning the AI service into one increasingly fragile universal rules engine.

The central ApexFormz principle becomes:

One body-understanding core. Many expert sports brains. One personalized athlete intelligence layer.